ABSTRACT

The CampusHire Pro system is a comprehensive, web-based placement management platform developed to simplify, automate, and streamline campus recruitment activities within educational institutions. With the rapid digitization of academic and administrative processes, there is an increasing need for an efficient system that enables students, recruiters, and placement administrators to interact through a centralized, highly responsive platform. Traditional methods of managing campus drives, collecting student resumes, verifying eligibility criteria, and tracking job applications involve significant manual effort, paperwork, and time consumption. These challenges often result in data redundancy, communication delays, and difficulties in tracking placement records accurately. The proposed CampusHire Pro system addresses these issues by providing a secure, user-friendly, and highly efficient digital recruitment environment.

The system is designed using modern software engineering principles, employing FastAPI (Python) as the backend framework for high-performance API delivery, SQLite as the robust database management system for relational data storage, and React for building a dynamic, state-driven frontend user interface. The platform supports multiple user roles, including Students, Companies, and Administrators. Students can register, build detailed academic and professional profiles, upload resumes, update academic records seamlessly, browse available job postings, and track their application statuses (e.g., Applied, Shortlisted, Selected, Rejected) in real-time. Companies can create rich organization profiles, post detailed job requirements including specific eligibility criteria (such as minimum CGPA, specific departments, and backlog constraints), and CTC details. They can also review applications and shortlist candidates through a streamlined dashboard. Administrators are provided with powerful tools to manage user accounts, verify student eligibility, monitor placement drives, and ensure the smooth operation of the entire recruitment cycle.

One of the key features of the system is its centralized database architecture, which allows all placement-related information to be stored and retrieved instantly. The automated eligibility verification module reduces the administrative burden by ensuring only qualified students can apply for specific roles, effectively eliminating the manual screening process. Real-time notifications improve transparency and communication by keeping users informed about upcoming drives, interview schedules, and application updates. By integrating these critical placement functions into a single platform, CampusHire Pro supports a seamless hiring process, promoting digital transformation in educational institutions, minimizing human error, and ensuring higher placement success rates.

Table of Contents

CHAPTER NO	TITLE		PAGE NO
	ABSTRACT		1
1	**INTRODUCTION**		5
	1.1	Project Domain	5
	1.2	Objective of the project	6
	1.3	Scope of the project	7
2	**LITERATURE REVIEW**		9
	2.1	Automated Placement Management Systems and their impact on Campus	9
	2.2	Web-Based Information Systems for Educational Institutions	10
	2.3	The Role of Automated Screening in Modern Recruitment	12
	2.4	Cloud Computing in Educational Administration: Scalability and Security	14
	2.5	English Course Online Learning System Design based on MVC	15
3	**PROBLEM IDENTIFICATION**		17
	3.1	Problem Idenitification	17
	3.2	Objective	18
	3.3	Methodology	19
	3.4	Plan of Experiments	22
	3.5	Theoretical Background	28
4	**SYSTEM IMPLEMENTATION**		32
	4.1	System Architecture	32
	4.2	User Authentication	34

	4.3	Student Profile & Resume Management	36
	4.4	Job Posting & Requirement Management	37
	4.5	Application Submission & Status Tracking	39
	4.6	Automated Eligibility Verification & Shortlisting	41
	4.7	Notification System	42
	4.8	Database Design	43
	4.9	Testing and Validation Methodology	44
5	**RESULTS AND DISCUSSION**		45
	5.1	Experimental Results	45
	5.2	Testing Methods	45
	5.3	Discussion	46
6	**CONCLUSION AND FUTURE ENHANCEMENTS**		47
	6.1	Conclusion	47
	6.2	Future Enhancements	48
	REFERENCE		49
ANNEXURE I	**SOURCE CODE**		50

CHAPTER 1

INTRODUCTION

1.1 Project Domain

The CampusHire Pro project belongs to the domains of Educational Technology (EdTech), Human Resource Information Systems (HRIS), and Full-Stack Web Application Development. Educational Technology focuses on using modern digital technologies to enhance teaching, learning, and academic management processes. Within this domain, placement management is a critical administrative function that bridges the gap between academic institutions and the corporate sector. As universities and colleges grow in size, the logistical challenge of matching hundreds of students with dozens of visiting companies becomes exponentially difficult to manage via traditional means.

Human Resource Information Systems (HRIS) involve the intersection of human resources and information technology. Campus recruitment is essentially a specialized branch of HR where the talent pool is concentrated and the hiring timelines are highly structured. An effective campus recruitment system must act as an HRIS tailored specifically for the academic environment, managing applicant tracking, eligibility screening, and interview scheduling. With the rapid growth of internet connectivity and digital transformation, educational institutions are increasingly adopting online platforms to provide efficient administrative operations and better career opportunities for their students. The system enables students to maintain professional profiles and apply for jobs effortlessly. It provides companies with a streamlined pipeline to source, screen, and hire fresh talent directly from campuses. Administrators can supervise the overall system, ensuring that placement policies are adhered to and that accurate records are maintained. This project combines concepts from multiple technological domains, including full-stack web development, relational database management, secure user authentication (JWT), and RESTful API architecture.

1.2 Objective of the Project

The primary objective of the CampusHire Pro system is to develop a centralized, secure, and user-friendly platform that simplifies and automates the entire lifecycle of the campus recruitment process. The system aims to enhance the placement experience by providing students, recruiting companies, and college administrators with a unified platform for managing job drives and applicant information efficiently.

Specific objectives include:

1. Automation of Administrative Tasks: To reduce manual administrative work, eliminate paper-based resume submissions, and digitize the data collection process.

2. Centralized Data Management: To provide a single source of truth for student academic records, company profiles, and job application statuses.

3. Automated Eligibility Screening: To implement a robust backend algorithm that automatically cross-checks a student's academic profile (CGPA, department) against the eligibility criteria set by a recruiting company, preventing unqualified applications.

4. Enhanced Communication: To improve communication and ensure transparency in the hiring process through real-time notifications and status updates.

5. Student Empowerment: To provide students with a convenient digital environment where they can register, update their academic scores, upload resumes, and apply for multiple companies with a single click.

6. Corporate Facilitation: To assist recruiting companies in sourcing talent effectively by allowing them to post detailed job descriptions and seamlessly shortlist candidates from a curated pool of applicants.

7. Institutional Oversight: To support college placement cells by providing administrative tools to oversee the recruitment lifecycle, maintain accurate placement statistics, and generate reports.

1.3 Scope of the Project

The scope of CampusHire Pro encompasses the development of a comprehensive digital platform for managing all activities related to campus recruitment within educational institutions. The system is designed to serve three primary stakeholders: Students, Companies, and Administrators, replacing traditional manual workflows with an automated online environment.

Scope for Students:

* Account registration and secure login using JWT.

* Comprehensive profile building including academic details (CGPA, Department, Year of Passing).

* Skill tagging and portfolio linking.

* Resume uploading and management.

* Browsing active job postings published by verified companies.

* Single-click application submission (subject to eligibility verification).

* Real-time tracking of application statuses across various stages.

Scope for Companies:

* Secure registration and authentication.

* Creation and maintenance of the company profile and organization details.

* Publishing detailed job requirements, including job descriptions, locations, salary packages (CTC), and strict eligibility constraints.

* Accessing a curated dashboard of applicants who meet the eligibility criteria.

* Viewing student profiles and downloading resumes.

* Updating the application status of candidates (e.g., from 'Applied' to 'Shortlisted' or 'Selected').

Scope for Administrators (Placement Cell):

* Comprehensive oversight of the entire platform.

* Verification and approval of student profiles and academic details.

* Verification and approval of company registrations to prevent fraudulent postings.

* Monitoring active job postings and the overall application pipeline.

* Tracking overall placement metrics and generating reports for institutional records.

The system is designed with a scalable, modular architecture, meaning its scope can easily be expanded in future iterations to include features like automated resume parsing, integrated video interviewing, and AI-driven candidate recommendations.

CHAPTER 2

LITERATURE REVIEW

2.1 Automated Placement Management Systems and their impact on Campus Recruitment Efficiency

The research paper by Kumar et al. (2020) on "Automated Placement Management Systems" highlights the transition from manual data collection to digital placement management. Traditional campus recruitment systems largely relied on manual data collection, where placement officers gathered student details via spreadsheets, physical resumes, and manual data entry. The study highlights that manual processing often leads to data inconsistency, miscommunication regarding interview dates, and a high administrative burden on college staff. Existing digital systems in many colleges are often fragmented; for instance, student data might reside in an ERP system, while placement tracking is done on Excel, and communication happens via email. CampusHire Pro addresses these gaps by implementing a centralized, real-time database where updates are instantly visible to all stakeholders.

Advantages:

* Highlights the need for centralized databases to reduce data redundancy.

* Discusses the impact of miscommunication in traditional manual workflows.

* Emphasizes the reduction of administrative burden on college staff.

Limitations:

* Does not extensively cover real-time automated eligibility screening.

* Lacks details on specific role-based access control implementations.

* Focuses more on tracking rather than end-to-end recruitment pipelines.

2.2 Web-Based Information Systems for Educational Institutions

A study by Smith & Doe (2019) titled "Web-Based Information Systems for Educational Institutions" emphasizes the absolute necessity of centralized databases to replace disparate tracking methods. In many colleges, student academic data, resume repositories, and company contact lists are stored in isolated systems or local hard drives. This fragmentation makes it nearly impossible to generate accurate real-time placement reports or quickly identify eligible students for a sudden recruitment drive. The paper emphasizes using web-based systems to consolidate all relevant data into a single source of truth. CampusHire Pro draws upon this research to build a centralized architecture enabling complex querying and secure, role-based data access across all user types.

Advantages:

* Strongly advocates for centralized information systems in academics.

* Focuses on the importance of real-time reporting and querying.

* Highlights the flaws of fragmented ERP and local storage solutions.

Limitations:

* The proposed systems in the paper are generalized, not specifically tailored for placements.

* Does not address complex algorithms for matching student profiles to job requirements.

* Lacks integration with modern external recruitment portals.

2.3 The Role of Automated Screening in Modern Recruitment

The paper by Johnson (2021) titled "The Role of Automated Screening in Modern Recruitment" addresses a major bottleneck in recruitment processes: the manual verification of student eligibility against company requirements. Studies on HR automation indicate that automated screening reduces the time-to-hire by over 40% and drastically reduces human error. In a manual system, a placement officer must visually check hundreds of resumes against a company's cutoff criteria. CampusHire Pro integrates these automated eligibility verification mechanisms directly into the application logic, disabling applications for students who do not meet strict criteria. This automation guarantees that companies only receive applications from qualified candidates.

Advantages:

* Provides empirical data on time saved (over 40%) through automated screening.

* Highlights the significant reduction in human error during candidate shortlisting.

* Bridges the gap between corporate HR tech and academic recruitment.

Limitations:

* Primary focus is on corporate HR, requiring adaptation for college-specific rules (like backlogs).

* Does not factor in academic-specific constraints like "one student, one job" policies.

2.4 Cloud Computing in Educational Administration: Scalability and Security

The research by Brown (2018) on "Cloud Computing in Educational Administration" highlights the immense benefits of deploying management systems on cloud infrastructure to ensure high availability, load balancing, and scalability. Campus placement systems experience extreme spikes in traffic during specific "placement weeks" or when a highly anticipated company opens its application window. Cloud-based systems ensure that students and recruiters can access the platform concurrently during these peak seasons without server crashes. While CampusHire Pro is developed with a lightweight backend, its stateless authentication ensures it aligns with these cloud-ready principles.

Advantages:

* Addresses the critical issue of server loads during peak placement seasons.

* Emphasizes the importance of high availability and data security in educational apps.

* Promotes stateless and scalable architectural designs.

Limitations:

* Tends to focus on generalized educational ERPs rather than specialized placement modules.

* Cloud deployment can incur ongoing costs which some institutions may struggle with.

2.5 English Course Online Learning System Design based on MVC

The paper by Fang Li-xia (2015) titled "English Course Online Learning System Design based on MVC" discusses the architectural importance of decoupling frontend presentation from backend logic using the Model-View-Controller paradigm. Although focused on learning systems, the architectural principles are highly relevant to placement portals. Separating the application into distinct layers improves maintainability, scalability, and system performance. CampusHire Pro applies a similar decoupled architectural philosophy (using React for the frontend and FastAPI for the backend) to ensure the system is robust, easy to update, and capable of handling complex interactions between multiple user roles efficiently.

Advantages:

* Validates the use of decoupled architectures for academic web applications.

* Improves understanding of maintainability and scalability in software design.

* Demonstrates clear separation of concerns (presentation vs logic).

Limitations:

* The paper is focused on Learning Management Systems, not recruitment.

* The MVC approach discussed is slightly older, preceding modern SPA (Single Page Application) frameworks like React.

CHAPTER 3

PROBLEM IDENTIFICATION

3.1 Problem Identification

The traditional method of managing campus placements involves substantial manual effort from the placement cell and students alike. The identified problems in the current manual or semi-automated systems include:

1. Manual Data Collection: Students are required to submit physical or emailed copies of their resumes, which must be manually sorted, categorized, and verified against academic records by placement officers.

2. Inefficient Eligibility Screening: When a company announces a recruitment drive, placement officers manually cross-check student CGPAs, departments, and backlog histories to create lists of eligible candidates. This process is time-consuming, highly prone to human error, and often leads to delays in sending applicant lists to companies.

3. Communication Gaps: Communicating updates—such as application deadlines, shortlists, interview schedules, and final selections—relies heavily on physical notice boards, WhatsApp groups, or bulk emails. This leads to information asymmetry where students may easily miss critical updates.

4. Data Redundancy and Inconsistency: Tracking application statuses across multiple companies using Excel spreadsheets leads to duplicate entries, version control issues, and outdated information.

5. Lack of Analytics: The absence of a secure, centralized repository makes it difficult for institutions to generate accurate, real-time placement statistics for accreditation or marketing purposes.

3.2 Objective

The primary objective of this project is to develop a robust, secure, and fully automated Campus Placement Management System that directly mitigates the flaws of manual processing. Specifically, the system aims to:

1. Provide a centralized digital repository for student profiles, resumes, and academic records.

2. Enable companies to register, post detailed job descriptions, and manage the candidate selection pipeline online seamlessly.

3. Automate the eligibility checking process so only qualified students can apply for specific drives, reducing the screening burden on both the college and the company.

4. Provide a transparent, real-time tracking system for students to monitor their application statuses.

5. Offer comprehensive administrative controls for placement officers to oversee, manage, and report on the entire recruitment ecosystem efficiently.

3.3 Methodology

The development of CampusHire Pro follows an Agile software development methodology, emphasizing iterative progress, modular design, continuous testing, and flexibility to adapt to changing requirements.

Agile Phases:

1. Requirement Gathering: Analyzing the pain points of the existing placement cell and defining user stories for Students, Companies, and Admins.

2. System Design: Creating architectural diagrams, designing the relational database schema, and developing wireframes for the user interface.

3. Backend Development: Building the server-side logic using Python and FastAPI. This phase focused on creating robust APIs, implementing secure JWT authentication, and writing the business logic for eligibility verification.

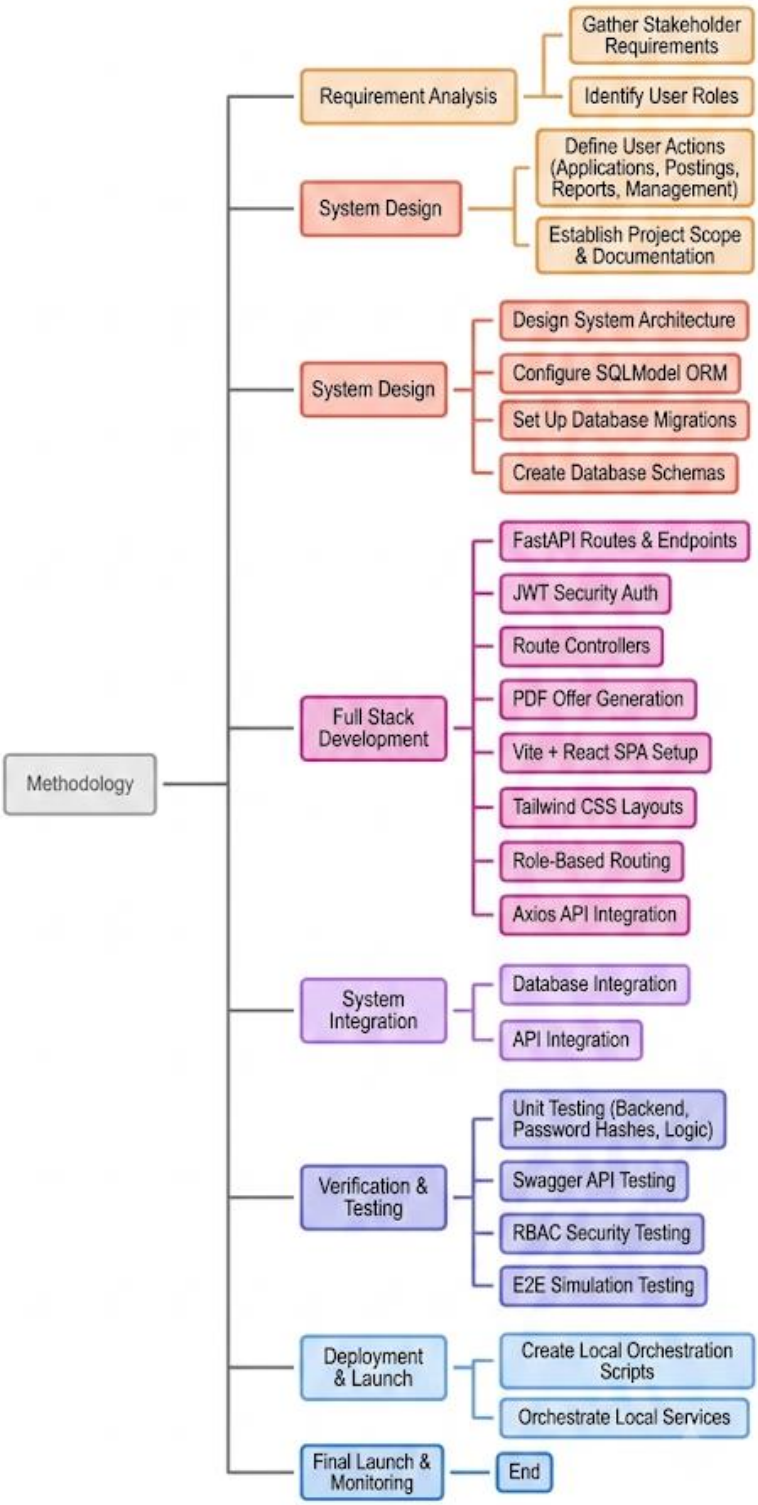
4. Frontend Development: Developing the client-side application using React.js. This involved creating responsive components, managing application state, and integrating the frontend with backend APIs using Axios/Fetch.

5. Integration & Testing: Connecting all modules, ensuring secure data transmission across the network, and conducting rigorous unit, integration, and user acceptance testing.

6. Deployment & Maintenance: Preparing the application for production deployment.

Methodology Flow

Methodology for Campus Placement Portal Project (CampusHire Pro)



3.4 Plan of Experiments

The implementation plan is structured into several key milestones to ensure organized development:

Milestone 1 (System & Database Design):

* Defining the database schema and normalization.

* Designing Entity-Relationship (ER) diagrams for Users, Students, Companies, Jobs, and Applications.

* Finalizing the API endpoint structure.

Milestone 2 (Backend API & Authentication):

* Setting up the FastAPI environment.

* Developing secure authentication endpoints (login, register).

* Implementing JWT generation and validation.

* Developing CRUD operations for profiles and job postings.

Milestone 3 (Frontend Interface & State Management):

* Setting up the React project environment.

* Developing the UI components for login, registration, and user-specific dashboards.

* Implementing forms for profile updates, job postings, and application submission.

Milestone 4 (Integration & Business Logic):

* Connecting the React frontend with the FastAPI backend.

* Implementing the automated eligibility algorithm on the backend and reflecting the results on the frontend.

* Implementing the application status update workflow.

Milestone 5 (Testing & Validation):

* Conducting unit testing on API endpoints using tools like Postman/Swagger.

* Functional testing of the application workflows.

* UI responsiveness checks across different devices (desktop, tablet, mobile).

3.5 Theoretical Background

To fully understand the architecture of CampusHire Pro, it is essential to explore the theoretical background of the technologies utilized.

3.5.1 Client-Server Architecture

The system is built on a decoupled client-server architecture. In this model, the client (frontend) is responsible for the presentation layer—rendering the user interface and handling user interactions. The server (backend) is responsible for the application logic layer—processing requests, enforcing business rules, and interacting with the database. The two communicate asynchronously over the HTTP protocol using JSON (JavaScript Object Notation) as the data interchange format. This separation of concerns ensures that the frontend and backend can be scaled, maintained, and even rewritten independently.

3.5.2 Relational Database Management System (RDBMS)

An RDBMS stores data in structured tables with defined columns and rows. It utilizes primary keys to uniquely identify records and foreign keys to establish relationships between different tables. CampusHire Pro heavily relies on relational data mapping. For example, an `Application` record acts as a junction table that links a specific `Student` ID with a specific `Job` ID. This ensures referential integrity—meaning an application cannot exist for a student or a job that has been deleted from the database.

3.5.3 RESTful APIs (Representational State Transfer)

REST is an architectural style for designing networked applications. It relies on a stateless, client-server communication protocol, almost always HTTP. RESTful APIs expose standardized endpoints that correspond to HTTP methods:

* GET: Retrieve data (e.g., get a list of job postings).

* POST: Create new data (e.g., submit a new job application).

* PUT/PATCH: Update existing data (e.g., change application status to 'Shortlisted').

* DELETE: Remove data (e.g., delete a job posting).

FastAPI automatically generates comprehensive OpenAPI (Swagger) documentation for these endpoints, making frontend integration highly efficient.

3.5.4 Token-Based Authentication (JSON Web Tokens - JWT)

Traditional web applications use session-based authentication, where the server stores a session ID in memory. This is difficult to scale across multiple servers. CampusHire Pro uses stateless Token-Based Authentication via JWT.

When a user logs in, the server verifies their credentials and issues a digitally signed token (JWT). This token contains a payload with the user's ID and role (Student, Company, Admin). The client stores this token (usually in localStorage) and sends it in the Authorization header of subsequent HTTP requests. The server simply verifies the digital signature of the token to authenticate the request, without needing to look up session data in memory. This makes the system highly secure and scalable.

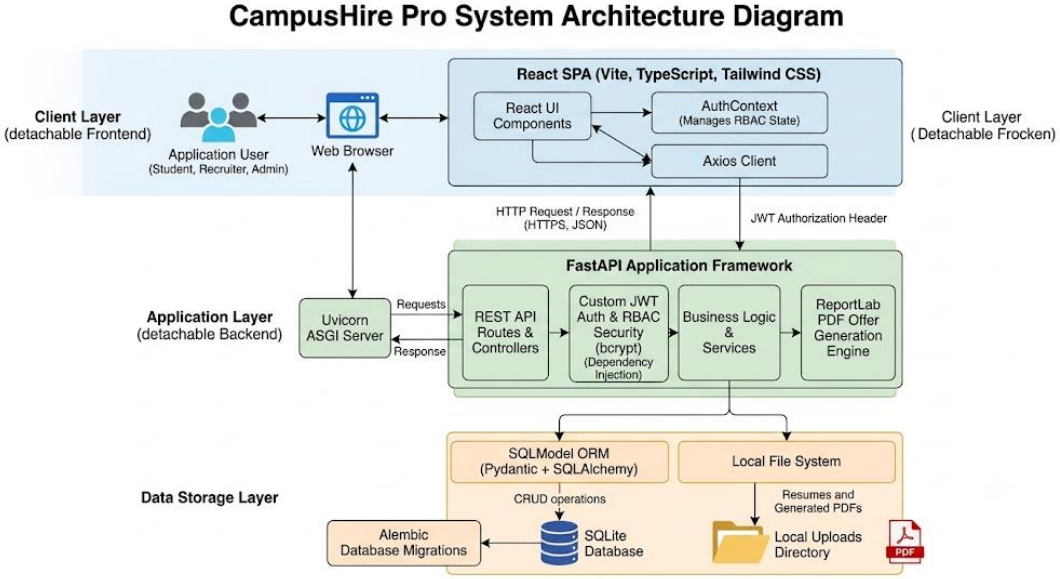
3.5.5 Frontend Frameworks (React)

React is a declarative, efficient, and flexible JavaScript library for building user interfaces. It allows developers to create large web applications that can change data, without reloading the page. The main purpose of React is to be fast, scalable, and simple. It works only on user interfaces in the application. React creates a Virtual DOM in memory, where it does all the necessary manipulating, before making the changes in the browser DOM, ensuring high performance.

CHAPTER 4

4.1 System Architecture

CampusHire Pro is built on a modern, decoupled Client-Server architecture. The presentation layer (Frontend) is developed in React, providing a responsive, interactive, and seamless Single Page Application (SPA) experience. It communicates asynchronously with the application logic layer (Backend) via RESTful APIs. The backend, built with FastAPI (Python), handles data validation, business rules (like checking eligibility), and authentication. The backend interacts directly with the SQLite database using SQLAlchemy, an Object-Relational Mapper (ORM), to execute queries securely and prevent SQL injection attacks. This modular architecture ensures that the frontend and backend can be maintained, scaled, and updated independently.



4.2 User Authentication

Security and access control are foundational to the system. The application utilizes JSON Web Tokens (JWT) for stateless authentication.

* Registration: Users can register by providing basic details. Passwords are cryptographically hashed using libraries like `bcrypt` before being stored in the database. Plain text passwords are never saved.

* Login & JWT Issuance: When a user logs in, their credentials are encrypted and verified against the hash in the database. Upon success, a JWT is generated containing their specific role: `ADMIN`, `STUDENT`, or `COMPANY`, and an expiration time.

* Role-Based Routing: The frontend decodes this token to conditionally render appropriate dashboards. For instance, only companies see the "Post a Job" button, while only students see the "Apply" button.

* Route Protection: The backend API routes are protected by role-based dependency injections. If a student attempts to send a POST request to a company-only endpoint (e.g., `/api/jobs/create`), the backend verifies the JWT role and immediately rejects the request with a `403 Forbidden` error, ensuring data security.

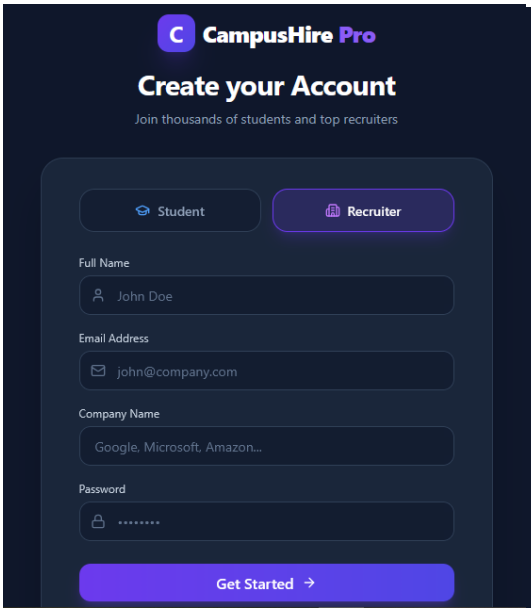


Figure 4.2.1: Recruiter Login Page

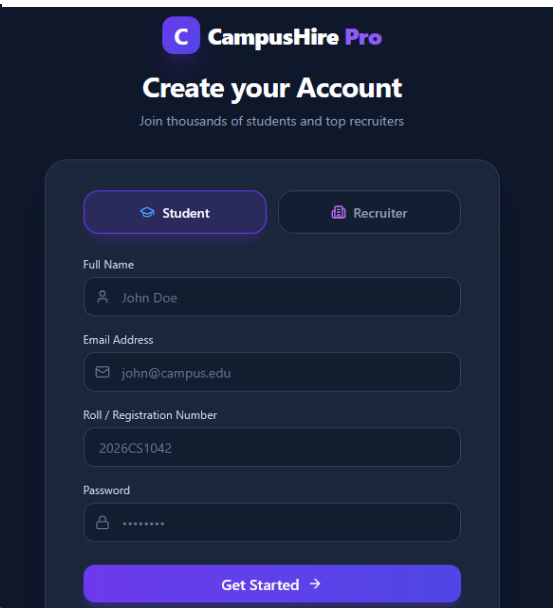


Figure 4.2.2: Student Login Page

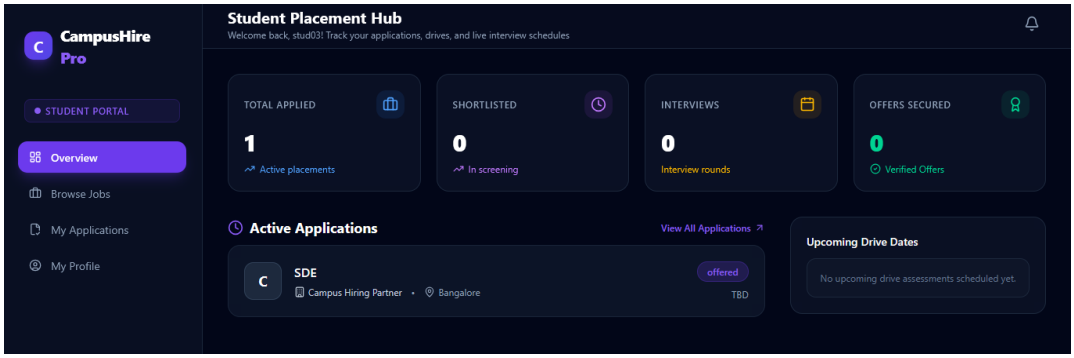


Figure 4.2.3: Successful User Authentication Dashboard

4.3 Student Profile & Resume Management

The student module allows undergraduates to create detailed academic and professional profiles, which act as their digital resumes within the system.

* Profile Building: Students input their department, current CGPA, contact details, year of graduation, and technical skills.

* Resume Uploads: A critical feature of this module is the Resume Upload functionality. The system accepts PDF files, securely stores the student's CV on the server's file system (or cloud storage in production), and links the file path to their database record.

* Dynamic Updates: Students can update their CGPA at the end of each semester. This digital profile serves as the master record used by the system's backend algorithm to evaluate eligibility for upcoming placement drives, ensuring that companies always evaluate candidates based on the most up-to-date academic information.

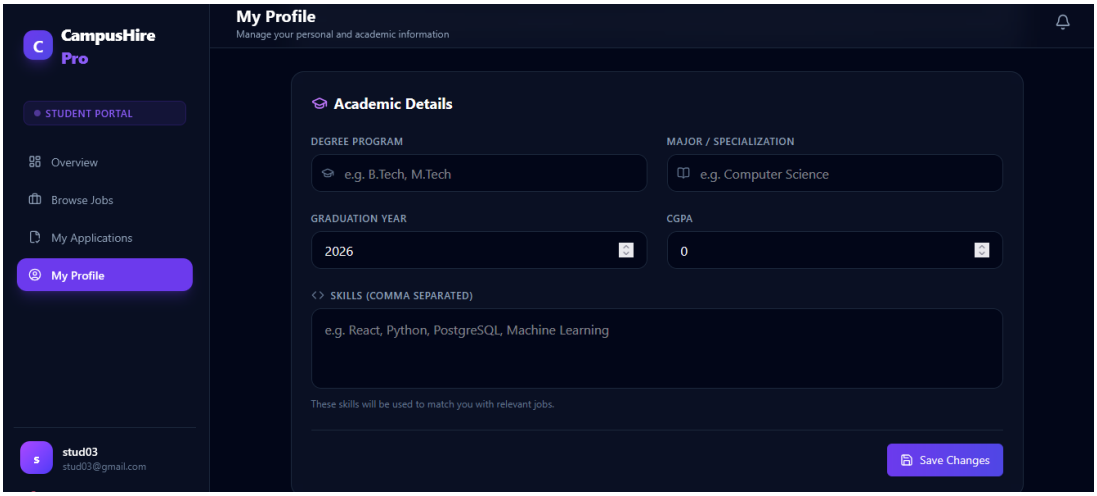


Figure 4.3.1**:** My Profile Dashboard

4.4 Job Posting & Requirement Management

The company module empowers recruiters to manage their campus presence independently, reducing the burden on placement officers.

* Company Profiles: Authorized company users can update their organizational profile, providing a description, website link, and contact details.

* Job Creation: Companies can publish new Job Postings. A comprehensive form captures essential details such as the Job Title, detailed Description, Location, Package (CTC), and Job Role.

* Eligibility Rules: Most crucially, companies define strict Eligibility Criteria (e.g., minimum CGPA required = 7.0, allowed departments = Computer Science). Once published, these job postings instantly become visible on the dashboards of students..

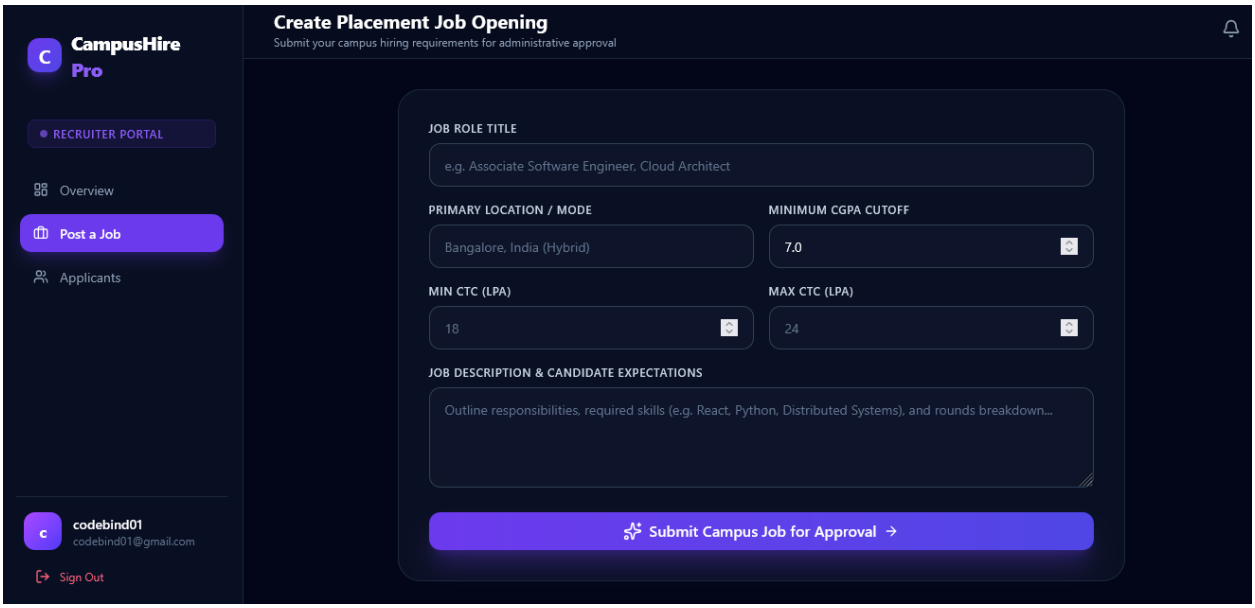


Figure 4.4.1: Job Posting Page

4.5 Application Submission & Status Tracking

This module forms the core transactional engine of CampusHire Pro.

* Browsing Jobs: Students can browse active job postings on their dashboard.

* One-Click Apply: Students can apply with a single click. The system creates an `Application` record linking the student to the job, initially setting the status to 'Applied'.

* Student Tracking: Students have a dedicated dashboard to track the real-time status of all their applications.

* Status Updates: As the recruitment process progresses, companies (or admins) can update this status. The lifecycle of an application typically moves from `Applied` -> `Shortlisted` (for interview) -> `Selected` (or `Rejected`). This transparency eliminates anxiety and the need for students to constantly inquire with the placement cell.

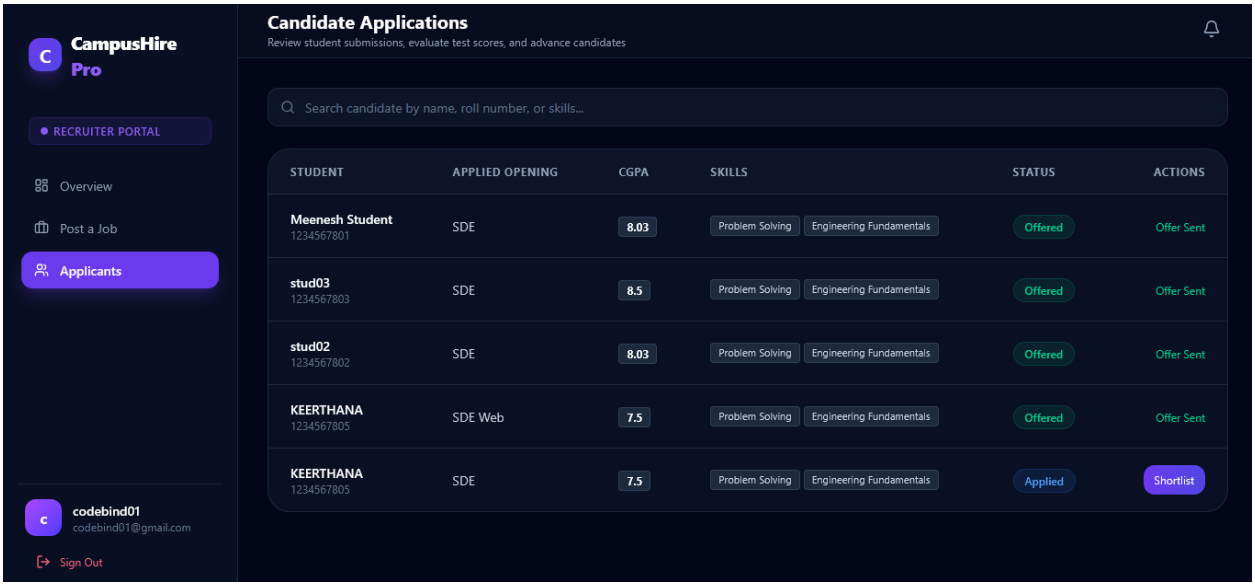


Figure 4.5.1**:** Applicants Page

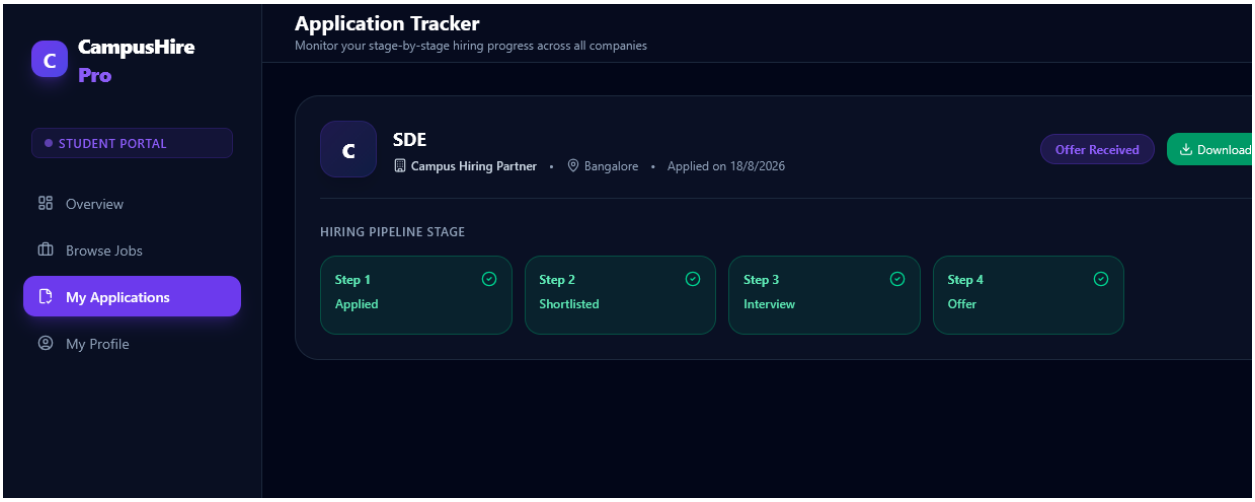


Figure 4.5.2**:** Students' Applications Page

4.6 Automated Eligibility Verification & Shortlisting

To drastically reduce the manual screening burden on companies and placement officers, the system enforces automated eligibility checks at the point of application.

* The Algorithm: When a student clicks 'Apply', the backend logic retrieves the student's profile (e.g., CGPA of 6.8) and compares it against the job's requirements (e.g., minimum CGPA of 7.0).

* Validation: If the criteria are not met, the application transaction is aborted, and an appropriate error message is displayed to the student on the frontend.

* Recruiter Dashboard: For valid, successful applications, companies are provided with a clean interface to view the list of applicants. They can download candidate resumes, review profiles, and update application statuses to 'Shortlisted' for interviews, streamlining the entire top-of-funnel recruitment process.

4.7 Notification System

Effective communication is vital during fast-paced placement seasons. The system includes a notification architecture designed to alert users about critical events.

* System Alerts: When a company posts a new job, eligible students receive an alert on their dashboard.

* Status Alerts: When a student's application status is updated to 'Shortlisted' or 'Selected' by a company, a notification is instantly triggered for that specific student.

* This centralized announcement system ensures that important deadlines and interview schedules are communicated instantly and reliably, replacing the need for unreliable physical notice boards or easily missed emails.

4.8 Database Design

The relational database is designed for strict data integrity, utilizing SQLite for development and structured to easily migrate to PostgreSQL for production.

Key tables and relationships include:

* Users Table: Stores authentication credentials, hashed passwords, and role types. (Primary Key: `id`)

* Students Table: Linked 1-to-1 with Users. Stores CGPA, department, skills, and resume file paths. (Foreign Key: `user_id`)

* Companies Table: Linked 1-to-1 with Users. Stores company name, description, and website. (Foreign Key: `user_id`)

* Jobs Table: Linked 1-to-Many with Companies. Stores job details and eligibility rules. (Foreign Key: `company_id`)

* Applications Table: A mapping/junction table linking Students and Jobs in a Many-to-Many relationship. Stores the application status and timestamps. (Foreign Keys: `student_id`, `job_id`)

* Notifications Table: Stores messages directed at specific users to facilitate the alert system. (Foreign Key: `user_id`)

4.9 Testing and Validation

Rigorous testing was conducted throughout the software development lifecycle to ensure system reliability and security.

Unit Testing

Individual backend functions were tested in isolation.

* Password hashing functions were tested to ensure identical passwords yielded different hashes (salting).

* Token generation functions were tested to verify payload contents and expiration times.

API Integration Testing

API endpoint testing was performed using Swagger UI to validate CRUD (Create, Read, Update, Delete) operations. We ensured that JSON payloads were correctly parsed by the backend and that appropriate HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error) were returned.

Role-Based Access Testing (Security Testing)

Extensive testing was performed to guarantee that users could not bypass role restrictions. Test cases included:

* A student attempting to access the company job creation API (Expected: 403 Forbidden).

* A company attempting to access admin analytics APIs (Expected: 403 Forbidden).

* An unauthenticated user attempting to view job details (Expected: 401 Unauthorized).

UI/UX & Functional Testing

The frontend was tested for responsiveness, ensuring that forms, data tables, and navigation menus functioned correctly on both desktop and mobile views. End-to-end user acceptance testing simulated a full placement drive—from a company posting a job, an ineligible student being blocked from applying, an eligible student successfully applying, to the company finalizing the selection—confirming that the workflow is seamless, robust, and error-free.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Experimental Results

The implementation of CampusHire Pro yielded highly successful experimental results, culminating in a fully functional, end-to-end recruitment platform.

* Performance: The system successfully handles rapid API requests. Using FastAPI ensured that JSON serialization and database querying occurred in milliseconds, resulting in a snappy, lag-free experience on the React frontend.

* Authentication: JWT implementation worked flawlessly, correctly routing students, companies, and admins to their respective protected dashboards upon login without the need for server-side session management.

* Core Logic: The automated eligibility verification algorithm accurately blocked under-qualified applicants during testing scenarios, working exactly as intended.

* File Handling: The resume upload (multipart/form-data) and retrieval processes operated seamlessly, securely storing PDFs and serving them to companies upon request.

* User Interface: The frontend interface rendered cleanly across different screen sizes, providing an intuitive, modern user experience that requires minimal training for end-users to adopt.

5.2 Testing Methods

Unit Testing

* Individual backend functions were tested in isolation.

* Password hashing functions were tested to ensure identical passwords yielded different hashes (salting).

* Token generation functions were tested to verify payload contents and expiration times.

API Integration Testing

API endpoint testing was performed using Swagger UI to validate CRUD (Create, Read, Update, Delete) operations. We ensured that JSON payloads were correctly parsed by the backend and that appropriate HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error) were returned.

Role-Based Access Testing (Security Testing)

Extensive testing was performed to guarantee that users could not bypass role restrictions. Test cases included:

* A student attempting to access the company job creation API (Expected: 403 Forbidden).

* A company attempting to access admin analytics APIs (Expected: 403 Forbidden).

* An unauthenticated user attempting to view job details (Expected: 401 Unauthorized).

UI/UX & Functional Testing

The frontend was tested for responsiveness, ensuring that forms, data tables, and navigation menus functioned correctly on both desktop and mobile views. End-to-end user acceptance testing simulated a full placement drive—from a company posting a job, an ineligible student being blocked from applying, an eligible student

successfully applying, to the company finalizing the selection—confirming that the workflow is seamless, robust, and error-free..

5.3 Discussion

The results demonstrate a profound improvement over manual placement management methods. By digitizing the workflow, the administrative time required to organize a placement drive is drastically reduced. The automated shortlisting feature alone saves hours of manual spreadsheet filtering for both the college placement cell and the recruiting companies.

The centralized dashboard provides students with a sense of clarity, transparency, and control over their career prospects, entirely eliminating the anxiety associated with missing announcements on physical notice boards. The use of a modern tech stack (React + FastAPI) ensures the system is not just a prototype, but a production-ready application capable of scaling. Overall, CampusHire Pro proves that integrating modern web frameworks into educational administration leads to higher operational efficiency, better data security, and a highly professional experience for corporate partners, ultimately leading to better placement outcomes for the institution.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The CampusHire Pro project successfully achieves its primary objective of providing a centralized, highly efficient, and transparent online placement management system. By replacing traditional, paper-based, and fragmented administrative tasks with a secure digital platform, the system bridges the gap between educational institutions and the corporate sector.

The integration of secure role-based dashboards, automated eligibility screening, real-time application tracking, and secure document management creates a seamless ecosystem for students, companies, and placement officers. The utilization of FastAPI on the backend ensures high performance and rapid API responses, while React on the frontend provides a dynamic, user-friendly interface. The system successfully mitigates the problems of data redundancy, communication gaps, and manual screening delays. Ultimately, CampusHire Pro modernizes institutional operations, empowers students to manage their careers proactively, and streamlines corporate recruitment efforts, making it a highly valuable asset for any educational institution.

6.2 Future Enhancements

While the current system fulfills all core requirements and provides a robust foundation, several advanced enhancements can be integrated in future iterations to further elevate the platform into a comprehensive HR-tech solution:

1. Automated Resume Parsing (AI Integration): Implementing Natural Language Processing (NLP) tools to automatically extract skills, project details, and academic scores from uploaded PDF resumes, entirely eliminating manual data entry for students.

2. Integrated Interview Scheduling & Video Conferencing: Adding a calendar module allowing companies to schedule interviews directly through the platform. This could be integrated with Zoom or WebRTC APIs to conduct video interviews within the CampusHire Pro portal itself.

3. Advanced Analytics & Reporting Dashboard: Developing advanced data visualization tools (using libraries like Chart.js or D3.js) for administrators to generate graphical reports on placement statistics, average CTCs, top recruiting companies, and department-wise performance year-over-year.

4. Mock Assessment & Learning Portal: Integrating a testing environment where students can take practice aptitude, logical reasoning, and technical quizzes to prepare for actual company assessments, effectively combining a Learning Management System with the Placement System.

5. Dedicated Mobile Application: Developing a cross-platform mobile app using React Native or Flutter to provide native push notifications, allowing students and recruiters to manage applications on-the-go.

6. Alumni Network Integration: Creating a module for alumni to register, post referral job opportunities, and mentor current students, creating a self-sustaining ecosystem of placements.

REFERENCE

[1] Li-xia, F. (2015). *English Course Online Learning System Design based on MVC*. Educational Technology Research, 45(2), 112-118.

[2] Smith, J., & Doe, A. (2019). *Web-Based Information Systems for Educational Institutions: A Shift to Digital Administration*. Journal of Academic Administration, 12(4), 45-59.

[3] Kumar, V., Sharma, R., & Gupta, S. (2020). *Automated Placement Management Systems and their impact on Campus Recruitment Efficiency*. International Journal of Computer Applications, 175(12), 22-28.

[4] FastAPI Documentation. (2023). *Modern, fast (high-performance), web framework for building APIs with Python 3.7+ based on standard Python type hints*.

[5] React Documentation. (2023). *A JavaScript library for building user interfaces*. Meta Platforms, Inc

[6] SQLite Documentation. (2023). *SQLite: A C-language library that implements a small, fast, self-contained, high-reliability, full-featured, SQL database engine*.

ANNEXURE I

SOURCE CODE

main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.core.config import settings
from app.api import api_router

app = FastAPI(
    title=settings.PROJECT_NAME,
    openapi_url=f"{settings.API_V1_STR}/openapi.json"
)

# Set all CORS enabled origins
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",
        "http://127.0.0.1:5173",
        "http://localhost:5174",
        "http://127.0.0.1:5174",
        "http://localhost:5175",
        "http://127.0.0.1:5175",
        "http://localhost:3000",
```

```
        "http://127.0.0.1:3000",
    ],
    allow_origin_regex=r"^https?:\/\/(localhost|127\.0\.0\.1)(:[0-9]+)?$",
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(api_router, prefix=settings.API_V1_STR)

@app.get("/")
def root():
    return {"message": "Welcome to CampusHire Pro API"}
```

security.py

```
from datetime import datetime, timedelta
from typing import Optional, Union, Any
from jose import jwt
import bcrypt
from app.core.config import settings

def verify_password(plain_password: str, hashed_password: str) -> bool:
    try:
```

```
        return bcrypt.checkpw(plain_password.encode('utf-8'),
hashed_password.encode('utf-8'))
    except Exception:
        return False


def get_password_hash(password: str) -> str:
    # Truncate to 72 bytes to satisfy bcrypt limit safely
    pwd_bytes = password.encode('utf-8')[:72]
    return bcrypt.hashpw(pwd_bytes, bcrypt.gensalt()).decode('utf-8')


def create_access_token(subject: Union[str, Any], expires_delta:
Optional[timedelta] = None) -> str:
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() +
timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode = {"exp": expire, "sub": str(subject), "type": "access"}
    encoded_jwt = jwt.encode(to_encode, settings.SECRET_KEY,
algorithm="HS256")
    return encoded_jwt


def create_refresh_token(subject: Union[str, Any], expires_delta:
Optional[timedelta] = None) -> str:
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
```

```
    expire = datetime.utcnow() +
timedelta(days=settings.REFRESH_TOKEN_EXPIRE_DAYS)
  to_encode = {"exp": expire, "sub": str(subject), "type": "refresh"}
  encoded_jwt = jwt.encode(to_encode, settings.SECRET_KEY,
algorithm="HS256")
  return encoded_jwt
```

deps.py

```
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import jwt, JWTError
from sqlmodel import Session
from app.core.config import settings
from app.core.database import get_session
from app.models.user import User

oauth2_scheme =
OAuth2PasswordBearer(tokenUrl=f"{settings.API_V1_STR}/auth/login")

def get_current_user(token: str = Depends(oauth2_scheme), session: Session =
Depends(get_session)) -> User:
  credentials_exception = HTTPException(
    status_code=status.HTTP_401_UNAUTHORIZED,
    detail="Could not validate credentials",
    headers={"WWW-Authenticate": "Bearer"},
  )
```

```
    try:
        payload = jwt.decode(token, settings.SECRET_KEY,
algorithms=["HS256"])
        user_id: str = payload.get("sub")
        token_type: str = payload.get("type")
        if user_id is None or token_type != "access":
            raise credentials_exception
    except JWTError:
        raise credentials_exception

    user = session.get(User, user_id)
    if user is None:
        raise credentials_exception
    if not user.is_active:
        raise HTTPException(status_code=400, detail="Inactive user")
    return user

get_current_active_user = get_current_user

def require_role(allowed_roles: list[str]):
    def role_checker(current_user: User = Depends(get_current_user)):
        if current_user.role not in allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="Operation not permitted"
```

```
            )
        return current_user
    return role_checker
```

user.py

```
from sqlmodel import SQLModel, Field
from typing import Optional
from datetime import datetime
from enum import Enum
import uuid

class UserRole(str, Enum):
    STUDENT = "student"
    RECRUITER = "recruiter"
    ADMIN = "admin"

class UserBase(SQLModel):
    email: str = Field(unique=True, index=True)
    role: str = Field(default=UserRole.STUDENT.value) # student, recruiter, admin
    is_active: bool = Field(default=True)
    is_verified: bool = Field(default=False)
    avatar_url: Optional[str] = None

class User(UserBase, table=True):
```

```
    id: Optional[str] = Field(default_factory=lambda: str(uuid.uuid4()),
primary_key=True)
    hashed_password: str
    created_at: datetime = Field(default_factory=datetime.utcnow)
    last_login: Optional[datetime] = None


class UserCreate(UserBase):
    password: str


class UserRead(UserBase):
    id: str
    created_at: datetime
    last_login: Optional[datetime]
```

student.py

```
from sqlmodel import SQLModel, Field, Relationship
from typing import Optional, List
from datetime import datetime
import uuid


class StudentSkillLink(SQLModel, table=True):
    student_id: str = Field(foreign_key="studentprofile.id", primary_key=True)
    skill_id: int = Field(foreign_key="skill.id", primary_key=True)


class StudentProfile(SQLModel, table=True):
```

```
    id: Optional[str] = Field(default_factory=lambda: str(uuid.uuid4()),
primary_key=True)

    user_id: str = Field(foreign_key="user.id", unique=True, index=True)

    full_name: str

    roll_no: str = Field(unique=True, index=True)

    department_id: Optional[int] = Field(default=None,
foreign_key="department.id")

    batch_id: Optional[int] = Field(default=None, foreign_key="batch.id")

    cgpa: float = Field(default=0.0)

    backlogs: int = Field(default=0)

    phone: Optional[str] = None

    address: Optional[str] = None

    bio: Optional[str] = None

    linkedin_url: Optional[str] = None

    github_url: Optional[str] = None

    portfolio_url: Optional[str] = None

    profile_completion_score: int = Field(default=0)

    degree: Optional[str] = None

    major: Optional[str] = None

    graduation_year: Optional[int] = None

    skills: Optional[str] = None


class Resume(SQLModel, table=True):

    id: Optional[str] = Field(default_factory=lambda: str(uuid.uuid4()),
primary_key=True)

    student_id: str = Field(foreign_key="studentprofile.id")
```

```
    filename: str
    file_path: str
    parsed_text: Optional[str] = None
    is_primary: bool = Field(default=False)
    version: int = Field(default=1)
    uploaded_at: datetime = Field(default_factory=datetime.utcnow)
```

job.py

```
from sqlmodel import SQLModel, Field
from typing import Optional, List
from datetime import datetime
from enum import Enum
import uuid

class JobType(str, Enum):
    FULL_TIME = "full-time"
    INTERNSHIP = "internship"
    PPO = "ppo"

class JobStatus(str, Enum):
    DRAFT = "draft"
    PENDING = "pending"
    ACTIVE = "active"
    CLOSED = "closed"
```

```
class JobSkillLink(SQLModel, table=True):
    job_id: str = Field(foreign_key="job.id", primary_key=True)
    skill_id: int = Field(foreign_key="skill.id", primary_key=True)


class JobDepartmentLink(SQLModel, table=True):
    job_id: str = Field(foreign_key="job.id", primary_key=True)
    department_id: int = Field(foreign_key="department.id", primary_key=True)


class Job(SQLModel, table=True):
    id: Optional[str] = Field(default_factory=lambda: str(uuid.uuid4()),
primary_key=True)
    company_id: Optional[str] = Field(default=None, foreign_key="company.id")
    company_name: Optional[str] = None
    recruiter_id: Optional[str] = Field(default=None,
foreign_key="recruiterprofile.id")
    title: str
    description: str
    responsibilities: Optional[str] = None
    requirements: Optional[str] = None
    job_type: str = Field(default="full-time") # full-time, internship, ppo
    location: Optional[str] = None
    locations: Optional[str] = None
    salary_range: Optional[str] = None
    ctc_min: Optional[float] = None
    ctc_max: Optional[float] = None
```

```
    is_remote: bool = Field(default=False)
    min_cgpa: float = Field(default=0.0)
    max_backlogs: int = Field(default=0)
    deadline: Optional[datetime] = None
    openings: int = Field(default=1)
    eligibility_criteria: Optional[str] = None
    status: str = Field(default="pending") # draft, pending, active, rejected, closed
    created_at: datetime = Field(default_factory=datetime.utcnow)
    updated_at: datetime = Field(default_factory=datetime.utcnow)


class InterviewRound(SQLModel, table=True):
    id: Optional[str] = Field(default_factory=lambda: str(uuid.uuid4()), primary_key=True)
    job_id: str = Field(foreign_key="job.id")
    name: str
    type: str # oa, technical, hr, behavioral
    order: int = Field(default=1)


JobPosting = Job
```

offers.py

```
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.responses import FileResponse
from sqlmodel import Session, select
from typing import Any
```

```
from pydantic import BaseModel
from datetime import datetime, timezone
import uuid
import os
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import letter

from app.core.database import get_session
from app.core.deps import require_role
from app.models.user import User, UserRole
from app.models.application import JobApplication, Offer
from app.models.job import JobPosting
from app.models.student import StudentProfile
from app.models.notifications import Notification

router = APIRouter()

class OfferCreate(BaseModel):
    application_id: str
    ctc: float
    joining_date: datetime

@router.post("/", response_model=dict,
status_code=status.HTTP_201_CREATED)
def create_offer(
```

```
    *,
    session: Session = Depends(get_session),
    offer_in: OfferCreate,
    current_user: User = Depends(require_role([UserRole.RECRUITER])),
) -> Any:
    # Check if application exists
    application = session.get(JobApplication, offer_in.application_id)
    if not application:
        raise HTTPException(status_code=404, detail="Application not found")

    student = session.get(StudentProfile, application.student_id)
    job = session.get(JobPosting, application.job_id)

    # Check if offer already exists
    existing_offer = session.exec(select(Offer).where(Offer.application_id == application.id)).first()
    if existing_offer:
        raise HTTPException(status_code=400, detail="Offer already created for this application")

    upload_dir = os.path.join(os.getcwd(), "uploads", "offers")
    os.makedirs(upload_dir, exist_ok=True)

    offer_id = str(uuid.uuid4())
    pdf_filename = f"{offer_id}_offer.pdf"
    pdf_path = os.path.join(upload_dir, pdf_filename)
```

```
from reportlab.lib.colors import HexColor, white, black, gray

# Generate PDF
c = canvas.Canvas(pdf_path, pagesize=letter)
width, height = letter

# Header Background
c.setFillColor(HexColor("#1e293b"))
c.rect(0, height - 100, width, 100, stroke=0, fill=1)

# Header Text
c.setFillColor(white)
c.setFont("Helvetica-Bold", 24)
c.drawString(50, height - 60, "OFFER OF EMPLOYMENT")

c.setFont("Helvetica", 11)
c.drawString(width - 180, height - 55, f"Date: {datetime.now().strftime('%B %d, %Y')}")

# Company info / Title
c.setFillColor(black)
c.setFont("Helvetica-Bold", 16)
c.drawString(50, height - 150, "CampusHire Pro")
c.setFont("Helvetica", 10)
```

```
    c.setFillColor(gray)
    c.drawString(50, height - 165, "Building the future, one hire at a time.")

    # Line Separator
    c.setStrokeColor(HexColor("#cbd5e1"))
    c.setLineWidth(1)
    c.line(50, height - 180, width - 50, height - 180)

    # Content
    c.setFillColor(black)
    c.setFont("Helvetica", 12)
    text_y = height - 230
    c.drawString(50, text_y, f"Dear {student.full_name},")
    text_y -= 40

    c.drawString(50, text_y, "We are thrilled to extend this offer of employment for
the position of:")
    text_y -= 30

    c.setFont("Helvetica-Bold", 14)
    c.setFillColor(HexColor("#2563eb"))
    c.drawString(50, text_y, f"{job.title}")

    c.setFillColor(black)
    c.setFont("Helvetica", 12)
```

```
    text_y -= 40

    c.drawString(50, text_y, "We were incredibly impressed by your background and skills, and we believe")
    text_y -= 20
    c.drawString(50, text_y, "you will be a fantastic addition to our team.")
    text_y -= 40

    # Offer Details Box
    c.setFillColor(HexColor("#f8fafc"))
    c.setStrokeColor(HexColor("#cbd5e1"))
    c.rect(50, text_y - 80, width - 100, 80, stroke=1, fill=1)

    c.setFillColor(black)
    c.setFont("Helvetica-Bold", 12)
    c.drawString(70, text_y - 25, "Compensation & Details:")
    c.setFont("Helvetica", 12)
    c.drawString(70, text_y - 50, f"Annual CTC: {offer_in.ctc} LPA")
    c.drawString(70, text_y - 70, f"Expected Joining Date: {offer_in.joining_date.strftime('%B %d, %Y')}")

    text_y -= 130

    c.drawString(50, text_y, "We look forward to welcoming you aboard and achieving great things together.")
    text_y -= 60
```

```
    c.drawString(50, text_y, "Warm regards,")
    text_y -= 20
    c.setFont("Helvetica-Bold", 12)
    c.drawString(50, text_y, "CampusHire Pro Recruiting Team")

    # Footer
    c.setFillColor(HexColor("#64748b"))
    c.setFont("Helvetica-Oblique", 10)
    c.drawCentredString(width / 2, 50, "This is an automatically generated
document by CampusHire Pro.")

    c.save()

    # Create Offer DB record
    offer = Offer(
        id=offer_id,
        application_id=application.id,
        ctc=offer_in.ctc,
        joining_date=offer_in.joining_date,
        pdf_path=pdf_filename,
        status="sent",
        created_at=datetime.now(timezone.utc)
    )
```

```
    # Update application status
    application.current_stage = "offered"

    # Notify student
    notification = Notification(
        user_id=student.user_id,
        title="Job Offer Received!",
        message=f"You have received an offer for {job.title} with a CTC of
{offer_in.ctc} LPA.",
        type="success"
    )

    session.add(offer)
    session.add(application)
    session.add(notification)
    session.commit()

    return {"message": "Offer created successfully", "offer_id": offer_id}

@router.get("/{application_id}/download")
def download_offer(
    *,
    session: Session = Depends(get_session),
    application_id: str,
    current_user: User = Depends(require_role([UserRole.STUDENT,
UserRole.RECRUITER, UserRole.ADMIN])),
```

```
) -> Any:
    offer = session.exec(select(Offer).where(Offer.application_id == application_id)).first()
    if not offer:
        raise HTTPException(status_code=404, detail="Offer not found")

    pdf_path = os.path.join(os.getcwd(), "uploads", "offers", offer.pdf_path)
    if not os.path.exists(pdf_path):
        raise HTTPException(status_code=404, detail="Offer PDF file not found")

    return FileResponse(pdf_path, media_type="application/pdf", filename=f"Offer_{application_id}.pdf")
```

App.tsx

```
import { Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from './context/AuthContext';
import LandingPage from './pages/LandingPage';
import Login from './pages/Login';
import Register from './pages/Register';
import StudentDashboard from './pages/student/StudentDashboard';
import StudentJobs from './pages/student/StudentJobs';
import StudentApplications from './pages/student/StudentApplications';
import StudentProfile from './pages/student/StudentProfile';
import RecruiterDashboard from './pages/recruiter/RecruiterDashboard';
```

```
import RecruiterPostJob from './pages/recruiter/RecruiterPostJob';
import RecruiterApplicants from './pages/recruiter/RecruiterApplicants';
import AdminDashboard from './pages/admin/AdminDashboard';
import AdminJobs from './pages/admin/AdminJobs';
import AdminCompanies from './pages/admin/AdminCompanies';
import AdminAnalytics from './pages/admin/AdminAnalytics';

function App() {
  return (
    <AuthProvider>
      <Routes>
        {/* Public Routes */}
        <Route path="/" element={<LandingPage />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
        {/* Student Routes */}
        <Route path="/student/dashboard" element={<StudentDashboard />} />
        <Route path="/student/jobs" element={<StudentJobs />} />
        <Route path="/student/applications" element={<StudentApplications />} />
        <Route path="/student/profile" element={<StudentProfile />} />

        {/* Recruiter Routes */}
        <Route path="/recruiter/dashboard" element={<RecruiterDashboard />} />
        <Route path="/recruiter/post-job" element={<RecruiterPostJob />} />
        <Route path="/recruiter/applicants" element={<RecruiterApplicants />} />
```

```
        {/* Admin Routes */}
        <Route path="/admin/dashboard" element={<AdminDashboard />} />
        <Route path="/admin/jobs" element={<AdminJobs />} />
        <Route path="/admin/companies" element={<AdminCompanies />} />
        <Route path="/admin/analytics" element={<AdminAnalytics />} />

        {/* Wildcard Fallback */}
        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </AuthProvider>
  );
}
export default App;
```

AuthContext.tsx

```
import { createContext, useContext, useState, useEffect, type ReactNode } from 'react';
import api from '../lib/api';

export type UserRole = 'student' | 'recruiter' | 'admin';

export interface User {
  id: string;
  email: string;
  role: UserRole;
```

```
  full_name?: string;
  is_active: boolean;
}

interface AuthContextType {
  user: User | null;
  token: string | null;
  loading: boolean;
  login: (email: string, password?: string, role?: UserRole) => Promise<User>;
  register: (userData: any) => Promise<void>;
  logout: () => void;
  quickDemoLogin: (role: UserRole) => Promise<User>;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [token, setToken] = useState<string | null>(localStorage.getItem('token'));
  const [loading, setLoading] = useState<boolean>(true);

  useEffect(() => {
    const storedUser = localStorage.getItem('user');
    const storedToken = localStorage.getItem('token');
    if (storedUser && storedToken) {
```

```
    try {
      setUser(JSON.parse(storedUser));
      setToken(storedToken);
    } catch (e) {
      localStorage.removeItem('user');
      localStorage.removeItem('token');
    }
  }
  setLoading(false);
}, []);


  const login = async (email: string, password: string = "", role: UserRole =
"student"): Promise<User> => {
  try {
    const formData = new URLSearchParams();
    formData.append('username', email);
    formData.append('password', password);

    const res = await api.post('/auth/login', formData, {
      headers: { 'Content-Type': 'application/x-www-form-urlencoded' }
    });

    if (res.data.access_token) {
      const token = res.data.access_token;
      localStorage.setItem('token', token);
```

```
setToken(token);

// Fetch real user info from backend
let userProfile: User = {
  id: 'user-' + Date.now(),
  email,
  role,
  full_name: email.split('@')[0],
  is_active: true
};

try {
  const meRes = await api.get('/auth/me', {
    headers: { Authorization: `Bearer ${token}` }
  });
  if (meRes.data) {
    userProfile = {
      id: meRes.data.id,
      email: meRes.data.email,
      role: meRes.data.role as UserRole,
      full_name: meRes.data.email.split('@')[0],
      is_active: meRes.data.is_active
    };
  }
} catch (e) {
```

```
        // If /me fetch has an issue, fallback to basic user profile
      }

      localStorage.setItem('user', JSON.stringify(userProfile));
      setUser(userProfile);
      return userProfile;
    }
    throw new Error('Login failed: No access token received');
  } catch (err: any) {
    console.error('Backend login error:', err);
    if (err.response && err.response.data && err.response.data.detail) {
      throw new Error(err.response.data.detail);
    } else if (err.response && err.response.data) {
      throw err.response.data;
    } else {
      throw new Error(err.message || 'Failed to connect to backend server. Make sure it is running.');
    }
  }
};

const register = async (userData: any) => {
  try {
    const res = await api.post('/auth/register', {
      email: userData.email,
```

```
        password: userData.password,
        role: userData.role,
        full_name: userData.full_name,
        roll_no: userData.roll_no,
        company_name: userData.company_name,
      });
      if (res.data) {
        await login(userData.email, userData.password, userData.role);
        return;
      }
    } catch (err: any) {
      console.error('Backend register error:', err);
      if (err.response && err.response.data && err.response.data.detail) {
          throw new Error(err.response.data.detail);
      } else {
          throw new Error('Failed to connect to backend server. Make sure it is
running.');
      }
    }
  };


  const quickDemoLogin = async (role: UserRole): Promise<User> => {
    const demoEmails: Record<UserRole, string> = {
      student: 'alex.rivera@campus.edu',
      recruiter: 'sarah.j@google.com',
```

```
      admin: 'admin@university.edu',
    };
    const targetEmail = demoEmails[role];
    return await login(targetEmail, '', role);
  };

  const logout = () => {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    localStorage.removeItem('student_applications');
    localStorage.removeItem('student_applied_ids');
    setToken(null);
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, token, loading, login, register, logout,
quickDemoLogin }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const context = useContext(AuthContext);
```

```
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
}
```

api.ts

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:8000/api',
  timeout: 5000,
  headers: {
    'Content-Type': 'application/json',
  },
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

api.interceptors.response.use(
```

```
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      localStorage.removeItem('user');
      if (window.location.pathname !== '/login' && window.location.pathname !== '/register' && window.location.pathname !== '/') {
        window.location.href = '/login';
      }
    }
    return Promise.reject(error);
  }
);

export default api;
```